```
!ls
```

```
sample_data
```

```
# CELL 1 — Mount Google Drive & Create Project Directories
# ═══════════════════════════════════════════════════════════════
from google.colab import drive
import os

# Mount Drive
drive.mount('/content/drive')

# Define base save path
BASE_PATH = '/content/drive/MyDrive/CSET419_Lab11'

# Create all necessary subdirectories
dirs = [
    BASE_PATH,
    f'{BASE_PATH}/component1_reviews',
    f'{BASE_PATH}/component1_reviews/model',
    f'{BASE_PATH}/component1_reviews/outputs',
    f'{BASE_PATH}/component2_recipes',
    f'{BASE_PATH}/component2_recipes/model',
    f'{BASE_PATH}/component2_recipes/outputs',
    f'{BASE_PATH}/reports',
]

for d in dirs:
    os.makedirs(d, exist_ok=True)

print("□ Google Drive mounted successfully!")
print(f"□ Project folder: {BASE_PATH}")
print("\nDirectory structure created:")
for d in dirs:
    print(f"  ✓ {d.replace(BASE_PATH, 'MyDrive/CSET419_Lab11')}")
```

```
Mounted at /content/drive
□ Google Drive mounted successfully!
□ Project folder: /content/drive/MyDrive/CSET419_Lab11

Directory structure created:
  ✓ MyDrive/CSET419_Lab11
  ✓ MyDrive/CSET419_Lab11/component1_reviews
  ✓ MyDrive/CSET419_Lab11/component1_reviews/model
  ✓ MyDrive/CSET419_Lab11/component1_reviews/outputs
  ✓ MyDrive/CSET419_Lab11/component2_recipes
  ✓ MyDrive/CSET419_Lab11/component2_recipes/model
  ✓ MyDrive/CSET419_Lab11/component2_recipes/outputs
  ✓ MyDrive/CSET419_Lab11/reports
```

```python
# ═══════════════════════════════════════════════════════════════════
# CELL 2 — Install Required Libraries
# ═══════════════════════════════════════════════════════════════════
!pip install transformers datasets accelerate -q

print("□ Libraries installed:")
print("  • transformers  — GPT-2 model & tokenizer")
print("  • datasets      — Dataset handling")
print("  • accelerate    — PyTorch training acceleration")
print("  • torch         — PyTorch (pre-installed in Colab)")
```

```
□ Libraries installed:
  • transformers  — GPT-2 model & tokenizer
  • datasets      — Dataset handling
  • accelerate    — PyTorch training acceleration
  • torch         — PyTorch (pre-installed in Colab)
```

```python
# ═══════════════════════════════════════════════════════════════════
# CELL 3 — Imports & Device Configuration
# ═══════════════════════════════════════════════════════════════════
import torch
import math
import json
import os
from datetime import datetime

from transformers import (
    GPT2LMHeadModel,
    GPT2Tokenizer,
    Trainer,
    TrainingArguments,
    DataCollatorForLanguageModeling,
    set_seed
)
from datasets import Dataset

# ── Reproducibility ──────────────────────────────────────────────
set_seed(42)

# ── Device Detection ─────────────────────────────────────────────
DEVICE = "cuda" if torch.cuda.is_available() else "cpu"

print("=" * 50)
print("  CSET419 Lab 11 — Fine-Tuning GPT-2")
print("=" * 50)
print(f"\n  Device     : {DEVICE.upper()}")
print(f"□  PyTorch    : {torch.__version__}")

if DEVICE == "cuda":
    print(f"⚡  GPU        : {torch.cuda.get_device_name(0)}")
```

```
    print(f"□  VRAM       :
{torch.cuda.get_device_properties(0).total_memory / 1e9:.1f} GB")
else:
    print("⚠  No GPU found. Go to Runtime → Change runtime type → T4
GPU")

print(f"\n□  Random seed : 42 (set for reproducibility)")
print(f"□  Timestamp   : {datetime.now().strftime('%Y-%m-%d %H:%M:
%S')}")
```

```
====================================================
  CSET419 Lab 11 — Fine-Tuning GPT-2
====================================================

  Device     : CUDA
□  PyTorch    : 2.10.0+cu128
⚡  GPU        : Tesla T4
□  VRAM       : 15.6 GB

□  Random seed : 42 (set for reproducibility)
□  Timestamp   : 2026-03-30 14:22:11
```

```
# ═══════════════════════════════════════════════════════════════
# CELL 4 — PyTorch Utility Functions
# ═══════════════════════════════════════════════════════════════

def generate_text(model, tokenizer, prompt,
                  max_new_tokens=80, temperature=0.8,
                  top_k=50, top_p=0.92):
    """Generate text using pure PyTorch inference."""
    model.eval()
    model = model.to(DEVICE)

    inputs = tokenizer.encode(prompt, return_tensors='pt').to(DEVICE)

    with torch.no_grad():
        output_ids = model.generate(
            inputs,
            max_new_tokens=max_new_tokens,
            temperature=temperature,
            top_k=top_k,
            top_p=top_p,
            do_sample=True,
            repetition_penalty=1.2,          # Avoid loops
            pad_token_id=tokenizer.eos_token_id,
        )

    return tokenizer.decode(output_ids[0], skip_special_tokens=True)


def compute_perplexity(trainer):
```

```python
    """Compute perplexity from eval loss — lower is better."""
    results = trainer.evaluate()
    ppl = math.exp(results["eval_loss"])
    return ppl, results


def tokenize_corpus(corpus, tokenizer, max_length=128):
    """Tokenize a list of strings into a HuggingFace Dataset."""
    dataset = Dataset.from_dict({'text': corpus})
    tokenized = dataset.map(
        lambda x: tokenizer(
            x['text'], truncation=True,
            max_length=max_length, padding='max_length'
        ),
        batched=True, remove_columns=['text']
    )
    return tokenized.train_test_split(test_size=0.15, seed=42)


def save_outputs_to_drive(data, filename, folder):
    """Save a dictionary as JSON to Google Drive."""
    path = f"{folder}/{filename}"
    with open(path, 'w') as f:
        json.dump(data, f, indent=2)
    print(f"  □ Saved → {path}")
    return path


def build_training_args(output_dir, epochs=15):
    """Build standard TrainingArguments for both components."""
    return TrainingArguments(
        output_dir=output_dir,
        num_train_epochs=epochs,
        per_device_train_batch_size=4,
        per_device_eval_batch_size=4,
        learning_rate=5e-5,
        weight_decay=0.01,
        warmup_steps=50,
        eval_strategy='epoch',
        logging_steps=10,
        save_strategy='no',
        fp16=torch.cuda.is_available(),
        report_to="none",
    )


print("□ Utility functions defined:")
print("  • generate_text()         — PyTorch inference with sampling")
print("  • compute_perplexity()    — Eval loss → PPL")
print("  • tokenize_corpus()       — Dataset prep & split")
```

```
print("  • save_outputs_to_drive() — JSON export to Drive")
print("  • build_training_args()   — Standard hyperparameters")
```

```
 Utility functions defined:
  • generate_text()         — PyTorch inference with sampling
  • compute_perplexity()    — Eval loss → PPL
  • tokenize_corpus()       — Dataset prep & split
  • save_outputs_to_drive() — JSON export to Drive
  • build_training_args()   — Standard hyperparameters
```

```
# ═══════════════════════════════════════════════════════════════════
# CELL 5 — Component I: Load GPT-2 for Product Reviews
# ═══════════════════════════════════════════════════════════════════
print(" Loading GPT-2 model and tokenizer...")

tokenizer_c1 = GPT2Tokenizer.from_pretrained('gpt2')
model_c1     = GPT2LMHeadModel.from_pretrained('gpt2')

# Fix: GPT-2 has no pad token — set it to eos
tokenizer_c1.pad_token                = tokenizer_c1.eos_token
model_c1.config.pad_token_id          = tokenizer_c1.eos_token_id

model_c1 = model_c1.to(DEVICE)

# ── Model stats ──────────────────────────────────────────────
total_params   = sum(p.numel() for p in model_c1.parameters())
trainable_params = sum(p.numel() for p in model_c1.parameters()
                       if p.requires_grad)

print("\n GPT-2 Loaded (Component I — Reviews)")
print(f"  Model          : GPT-2 (Base)")
print(f"  Device         : {DEVICE.upper()}")
print(f"  Total params   : {total_params:,}")
print(f"  Trainable      : {trainable_params:,}")
print(f"  Vocab size     : {tokenizer_c1.vocab_size:,}")
print(f"  Context window : 1,024 tokens")
```

```
 Loading GPT-2 model and tokenizer...

/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/
_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your
settings tab (https://huggingface.co/settings/tokens), set it as
secret in your Google Colab and restart your session.
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to
access public models or datasets.
  warnings.warn(
Warning: You are sending unauthenticated requests to the HF Hub.
Please set a HF_TOKEN to enable higher rate limits and faster
```

```
downloads.
WARNING:huggingface_hub.utils._http:Warning: You are sending
unauthenticated requests to the HF Hub. Please set a HF_TOKEN to
enable higher rate limits and faster downloads.
```


```
{"model_id":"925ff88fee7d44cf96e880e9cf793378","version_major":2,"vers
ion_minor":0}

{"model_id":"5c62d67ee5b24b1489e96f04c594ea36","version_major":2,"vers
ion_minor":0}

{"model_id":"50f04655254f4dd3a9f07e87403761de","version_major":2,"vers
ion_minor":0}

{"model_id":"adb488c5e0004082a8eeb89aabc6c431","version_major":2,"vers
ion_minor":0}

{"model_id":"152f160efaa44e698ed29fb315c2119b","version_major":2,"vers
ion_minor":0}

{"model_id":"92ad0c709bfc48cb978ff4a1ff23d82a","version_major":2,"vers
ion_minor":0}

{"model_id":"97cc45b57f5e46b4a0b9b57c5440ab01","version_major":2,"vers
ion_minor":0}
```


```
GPT2LMHeadModel LOAD REPORT from: gpt2
Key                    | Status     |  |
----------------------+------------+--+-
h.{0...11}.attn.bias | UNEXPECTED |  |

Notes:
- UNEXPECTED    :can be ignored when loading from different
task/architecture; not ok if you expect identical arch.
```


```
{"model_id":"57dc402de8024b9a8306f30cabf4b8d2","version_major":2,"vers
ion_minor":0}
```


```
 GPT-2 Loaded (Component I — Reviews)
  Model          : GPT-2 (Base)
  Device         : CUDA
  Total params   : 124,439,808
  Trainable      : 124,439,808
  Vocab size     : 50,257
  Context window : 1,024 tokens
```

```
# ═══════════════════════════════════════════════════════════════════
# CELL 6 — Component I: Baseline Reviews (Before Fine-Tuning)
# ═══════════════════════════════════════════════════════════════════
REVIEW_PROMPTS = [
    'This product is',
```

```
    'I bought this phone and',
    'The quality of this item',
    'I highly recommend',
]

print("=" * 60)
print("  BASELINE REVIEWS — Before Fine-Tuning")
print("=" * 60)

baseline_c1 = {}
for i, prompt in enumerate(REVIEW_PROMPTS, 1):
    output = generate_text(model_c1, tokenizer_c1, prompt)
    baseline_c1[prompt] = output
    print(f"\n[{i}] Prompt   : {prompt}")
    print(f"    Generated: {output[:180]}")

# Save baseline to Drive
save_outputs_to_drive(
    {"stage": "baseline", "component": 1, "outputs": baseline_c1},
    "c1_baseline_reviews.json",
    f"{BASE_PATH}/component1_reviews/outputs"
)
print("\n□ Observation: Baseline outputs are generic — news/Wikipedia
style text")
```

```
The attention mask is not set and cannot be inferred from input
because pad token is same as eos token. As a consequence, you may
observe unexpected behavior. Please pass your input's `attention_mask`
to obtain reliable results.

============================================================
  BASELINE REVIEWS — Before Fine-Tuning
============================================================

[1] Prompt   : This product is
    Generated: This product is designed to be used in conjunction with
any other device and may contain nicotine, but not if you have read
the label or are at all familiar.


: The Product Descri

[2] Prompt   : I bought this phone and
    Generated: I bought this phone and the back is amazing. It's
perfect for people who are looking to carry on with their family life,
whether it be from home or in a car: if you're new there ma

[3] Prompt   : The quality of this item
    Generated: The quality of this item is excellent. The paint job
was very smooth and finished on all four sides without any rust or
```

```
discoloration, even though it had no air pockets in the bott

[4] Prompt   : I highly recommend
    Generated: I highly recommend you take this with a grain of salt
and put it to good use!
What's the best thing about "The Art of Cooking" (aka, cook your own
pasta)? It takes less than half a
  □ Saved →
/content/drive/MyDrive/CSET419_Lab11/component1_reviews/outputs/c1_bas
eline_reviews.json

□ Observation: Baseline outputs are generic — news/Wikipedia style
text

# ═══════════════════════════════════════════════════════════════════
# CELL 7 — Component I: Dataset Preparation & Fine-Tuning
# ═══════════════════════════════════════════════════════════════════

REVIEW_CORPUS = [
    'this phone has an amazing battery life and the camera quality is
outstanding for the price.',
    'i bought this laptop for college and it handles all my
assignments and coding projects perfectly.',
    'the sound quality of these headphones is incredible with deep
bass and clear vocals.',
    'this smartwatch tracks my steps accurately and the heart rate
monitor is very reliable.',
    'great wireless earbuds with noise cancellation that blocks out
all background sound.',
    'the keyboard feels very comfortable for long typing sessions and
the backlight is a nice touch.',
    'this portable charger saved me during travel and it charges my
phone three times on a single charge.',
    'the tablet screen is bright and colorful which makes watching
movies a great experience.',
    'i love this fitness tracker because it motivates me to reach my
daily exercise goals.',
    'this bluetooth speaker is compact but delivers surprisingly loud
and clear audio.',
    'the delivery was fast and the product was packed securely with no
damage at all.',
    'excellent value for money and the build quality feels premium
despite the affordable price.',
    'the customer service team was very helpful when i had questions
about the product features.',
    'this camera takes stunning photos in low light and the video
recording quality is very smooth.',
    'i have been using this product for three months and it still
works perfectly like day one.',
    'the design is sleek and modern and it looks great on my desk next
```

```
to my other gadgets.',
    'easy to set up right out of the box and the instructions were
clear and simple to follow.',
    'highly recommend this product to anyone looking for quality and
reliability at a fair price.',
    'the software updates keep adding new features which makes this
purchase even more worthwhile.',
    'best purchase i made this year and i would definitely buy from
this brand again.',
]

# ── Tokenize ──────────────────────────────────────────────────
print("□ Tokenizing review corpus...")
split_c1 = tokenize_corpus(REVIEW_CORPUS, tokenizer_c1)
print(f"  Train samples : {len(split_c1['train'])}")
print(f"  Eval  samples : {len(split_c1['test'])}")

# ── Data Collator ─────────────────────────────────────────────
collator_c1 = DataCollatorForLanguageModeling(
    tokenizer=tokenizer_c1, mlm=False   # Causal LM, not masked
)

# ── Training Arguments ────────────────────────────────────────
args_c1 = build_training_args(
    output_dir=f'/content/gpt2-reviews',
    epochs=15
)

# ── Trainer ───────────────────────────────────────────────────
trainer_c1 = Trainer(
    model=model_c1,
    args=args_c1,
    train_dataset=split_c1['train'],
    eval_dataset=split_c1['test'],
    data_collator=collator_c1,
)

print("\n□ Starting fine-tuning (15 epochs)...")
print("   Hyperparameters:")
print("   • Epochs    : 15")
print("   • Batch     : 4")
print("   • LR        : 5e-5")
print("   • Warmup    : 50 steps")
print("   • FP16      : True (GPU)\n")

trainer_c1.train()
```

```
□ Tokenizing review corpus...
```

{"model_id":"e4470f29430a45b98026330ad2349203","version_major":2,"version_minor":0}

```
  Train samples : 17
  Eval  samples : 3

 Starting fine-tuning (15 epochs)...
   Hyperparameters:
   • Epochs    : 15
   • Batch     : 4
   • LR        : 5e-5
   • Warmup    : 50 steps
   • FP16      : True (GPU)


`loss_type=None` was set in the config but it is unrecognized. Using
the default loss: `ForCausalLMLoss`.

<IPython.core.display.HTML object>

TrainOutput(global_step=75, training_loss=1.9250287294387818,
metrics={'train_runtime': 12.7245, 'train_samples_per_second': 20.04,
'train_steps_per_second': 5.894, 'total_flos': 16657367040000.0,
'train_loss': 1.9250287294387818, 'epoch': 15.0})
```

```python
# ═══════════════════════════════════════════════════════════════════
# CELL 8 — Component I: Evaluate, Compare & Save
# ═══════════════════════════════════════════════════════════════════

# ── Perplexity ──────────────────────────────────────────────
ppl_c1, eval_res_c1 = compute_perplexity(trainer_c1)
print(f"  Perplexity (Component I): {ppl_c1:.2f}")
print(f"   (Lower = better domain fit. GPT-2 baseline PPL ~120–200 on
this data)")

# ── Compare Baseline vs Fine-Tuned ──────────────────────────
print("\n" + "=" * 65)
print("  COMPARISON: Baseline vs Fine-Tuned — Product Reviews")
print("=" * 65)

comparison_c1 = []
for prompt in REVIEW_PROMPTS:
    ft_out = generate_text(model_c1, tokenizer_c1, prompt)
    rec = {
        "prompt":    prompt,
        "baseline":  baseline_c1[prompt],
        "finetuned": ft_out
    }
    comparison_c1.append(rec)

    print(f"\n  Prompt   : {prompt}")
```

```
    print(f"  □ Baseline  : {baseline_c1[prompt][:140]}")
    print(f"  □ Fine-Tuned: {ft_out[:140]}")

# ── Save model to Drive ──────────────────────────────────────
model_save_path_c1 = f"{BASE_PATH}/component1_reviews/model"
print("\n\n□ Saving fine-tuned model to Drive...")
model_c1.save_pretrained(model_save_path_c1)
tokenizer_c1.save_pretrained(model_save_path_c1)
print(f"  □ Model saved → {model_save_path_c1}")

# ── Save outputs JSON ────────────────────────────────────────
save_outputs_to_drive(
    {
        "perplexity": round(ppl_c1, 2),
        "eval_loss":  round(eval_res_c1["eval_loss"], 4),
        "comparisons": comparison_c1
    },
    "c1_finetuned_reviews.json",
    f"{BASE_PATH}/component1_reviews/outputs"
)
```

```
<IPython.core.display.HTML object>

□ Perplexity (Component I): 24.74
   (Lower = better domain fit. GPT-2 baseline PPL ~120–200 on this
data)

====================================================================
  COMPARISON: Baseline vs Fine-Tuned — Product Reviews
====================================================================

□ Prompt   : This product is
  □ Baseline  : This product is designed to be used in conjunction
with any other device and may contain nicotine, but not if you have
read the label or are
  □ Fine-Tuned: This product is packed with features that make this
purchase even more worthwhile. The quality of the components makes
these purchases a rea

□ Prompt   : I bought this phone and
  □ Baseline  : I bought this phone and the back is amazing. It's
perfect for people who are looking to carry on with their family life,
whether it be from
  □ Fine-Tuned: I bought this phone and it handles all my calls
perfectly. The camera quality is outstanding for the price which makes
watching movies a gre

□ Prompt   : The quality of this item
  □ Baseline  : The quality of this item is excellent. The paint job
was very smooth and finished on all four sides without any rust or
```

```
discoloration, even
  □ Fine-Tuned: The quality of this item is exceptional for the price.
The build feels premium and features a sleek look that makes watching
movies even mor

□ Prompt   : I highly recommend
  □ Baseline  : I highly recommend you take this with a grain of salt
and put it to good use!
What's the best thing about "The Art of Cooking" (aka, cook yo
  □ Fine-Tuned: I highly recommend this product to anyone looking for
quality and reliability at a fair price.



□ Saving fine-tuned model to Drive...
```

{"model_id":"db8a5aee63ef4a73a0ec842400d384ab","version_major":2,"version_minor":0}

```
  □ Model saved →
/content/drive/MyDrive/CSET419_Lab11/component1_reviews/model
  □ Saved →
/content/drive/MyDrive/CSET419_Lab11/component1_reviews/outputs/c1_finetuned_reviews.json
```

{"type":"string"}

```
# ═══════════════════════════════════════════════════════════════════
# CELL 9 — Component II: Load FRESH GPT-2 (not the review model!)
# ═══════════════════════════════════════════════════════════════════
print("□ Loading a FRESH GPT-2 for recipe generation...")
print("   (This is a clean pre-trained model, not the review one)\n")

tokenizer_c2 = GPT2Tokenizer.from_pretrained('gpt2')
model_c2     = GPT2LMHeadModel.from_pretrained('gpt2')

tokenizer_c2.pad_token       = tokenizer_c2.eos_token
model_c2.config.pad_token_id = tokenizer_c2.eos_token_id

model_c2 = model_c2.to(DEVICE)

# Verify models are different objects in memory
print(f"□ Fresh GPT-2 loaded for Component II")
print(f"   model_c1 id: {id(model_c1)}")
print(f"   model_c2 id: {id(model_c2)}  ← different object ✓")
print(f"   Device     : {next(model_c2.parameters()).device}")
```

```
□ Loading a FRESH GPT-2 for recipe generation...
   (This is a clean pre-trained model, not the review one)
```

{"model_id":"8d4844a7ce594c3fa2b3e83ca3b2d7c3","version_major":2,"version_minor":0}

```
GPT2LMHeadModel LOAD REPORT from: gpt2
Key                   | Status     |  |
----------------------+------------+--+-
h.{0...11}.attn.bias | UNEXPECTED |  |

Notes:
- UNEXPECTED    :can be ignored when loading from different
task/architecture; not ok if you expect identical arch.

 Fresh GPT-2 loaded for Component II
   model_c1 id: 132618248858480
   model_c2 id: 132618115546960  ← different object ✓
   Device     : cuda:0
```

```python
# ═══════════════════════════════════════════════════════════════════
# CELL 10 — Component II: Baseline Recipes (Before Fine-Tuning)
# ═══════════════════════════════════════════════════════════════════
RECIPE_PROMPTS = [
    'To make butter chicken',
    'For pasta carbonara',
    'To prepare a chocolate cake',
    'The first step in making stir fry is',
]

print("=" * 60)
print("  BASELINE RECIPES — Before Fine-Tuning")
print("=" * 60)

baseline_c2 = {}
for i, prompt in enumerate(RECIPE_PROMPTS, 1):
    output = generate_text(model_c2, tokenizer_c2, prompt)
    baseline_c2[prompt] = output
    print(f"\n[{i}] Prompt   : {prompt}")
    print(f"    Generated: {output[:180]}")

save_outputs_to_drive(
    {"stage": "baseline", "component": 2, "outputs": baseline_c2},
    "c2_baseline_recipes.json",
    f"{BASE_PATH}/component2_recipes/outputs"
)
print("\n Observation: Baseline generates off-topic text, not cooking steps")
```

```
============================================================
  BASELINE RECIPES — Before Fine-Tuning
============================================================

[1] Prompt   : To make butter chicken
```

```
    Generated: To make butter chicken with sugar and water, add the
onions in a small saucepan. Bring to boil then saute for 2-3 minutes
until tender (about 1 minute per side). Meanwhile whisk to

[2] Prompt   : For pasta carbonara
    Generated: For pasta carbonara, we prefer the whole-wheat method.
This allows us to use less salt than a regular cup of hot water and
also for better results on high heat!
Ingredients For Cau

[3] Prompt   : To prepare a chocolate cake
    Generated: To prepare a chocolate cake for the oven, you will need
to find your pie crust. I used my own dough from our homemade bakery
and it was very easy! Make sure that this is an airtigh

[4] Prompt   : The first step in making stir fry is
    Generated: The first step in making stir fry is to make it
yourself and whisk until thick. Once the mixture has reached a simmer,
add garlic powder (or another salt) or ground pepper flakes a
  □ Saved →
/content/drive/MyDrive/CSET419_Lab11/component2_recipes/outputs/c2_bas
eline_recipes.json

□ Observation: Baseline generates off-topic text, not cooking steps

# ═══════════════════════════════════════════════════════════════════
# CELL 11 — Component II: Dataset Preparation & Fine-Tuning
# ═══════════════════════════════════════════════════════════════════

RECIPE_CORPUS = [
    'to make butter chicken start by marinating chicken pieces in
yogurt with turmeric chili powder and garam masala for one hour.',
    'heat butter in a pan and fry onions until golden brown then add
ginger garlic paste and cook for two minutes.',
    'add tomato puree and cook on low heat for ten minutes until the
oil separates from the masala.',
    'add the marinated chicken and cook on medium heat for fifteen
minutes until fully cooked.',
    'finish with fresh cream and kasuri methi and serve hot with naan
or steamed rice.',
    'for pasta carbonara boil spaghetti in salted water until al dente
and reserve half cup of pasta water.',
    'fry diced pancetta in olive oil until crispy and set aside.',
    'whisk together egg yolks parmesan cheese and black pepper in a
bowl.',
    'toss the hot pasta with pancetta and remove from heat then
quickly stir in the egg mixture.',
    'the residual heat will cook the eggs into a creamy sauce and
serve immediately with extra parmesan.',
    'to prepare vegetable stir fry heat sesame oil in a wok on high
```

```
heat.',
    'add sliced bell peppers broccoli florets and snap peas and toss
for three minutes.',
    'pour in soy sauce oyster sauce and a pinch of sugar and stir
well.',
    'add minced garlic and ginger and cook for one more minute until
fragrant.',
    'serve the stir fry over steamed jasmine rice and garnish with
sesame seeds.',
    'for chocolate chip cookies cream together butter and sugar until
light and fluffy.',
    'beat in eggs one at a time then add vanilla extract and mix
well.',
    'fold in flour baking soda and salt then gently stir in chocolate
chips.',
    'scoop dough onto a baking sheet and bake at 180 degrees for
twelve minutes until golden.',
    'let cookies cool on the tray for five minutes before transferring
to a wire rack.',
]

# ── Tokenize ──────────────────────────────────────────────────
print("□ Tokenizing recipe corpus...")
split_c2 = tokenize_corpus(RECIPE_CORPUS, tokenizer_c2)
print(f"  Train samples : {len(split_c2['train'])}")
print(f"  Eval  samples : {len(split_c2['test'])}")

collator_c2 = DataCollatorForLanguageModeling(tokenizer=tokenizer_c2,
mlm=False)

args_c2 = build_training_args(
    output_dir='/content/gpt2-recipes',
    epochs=15
)

trainer_c2 = Trainer(
    model=model_c2,
    args=args_c2,
    train_dataset=split_c2['train'],
    eval_dataset=split_c2['test'],
    data_collator=collator_c2,
)

print("\n□ Starting fine-tuning (Component II — Recipes)...")
trainer_c2.train()
```

```
□ Tokenizing recipe corpus...
```

{"model_id":"4a118c03a4ac49668ca0a18b5fe29026","version_major":2,"version_minor":0}

```
  Train samples : 17
  Eval  samples : 3

 Starting fine-tuning (Component II — Recipes)...

<IPython.core.display.HTML object>

TrainOutput(global_step=75, training_loss=2.0859599272410074,
metrics={'train_runtime': 7.3511, 'train_samples_per_second': 34.689,
'train_steps_per_second': 10.203, 'total_flos': 16657367040000.0,
'train_loss': 2.0859599272410074, 'epoch': 15.0})
```

```python
# ═══════════════════════════════════════════════════════════════════
# CELL 12 — Component II: Evaluate, Compare & Save
# ═══════════════════════════════════════════════════════════════════

ppl_c2, eval_res_c2 = compute_perplexity(trainer_c2)
print(f"  Perplexity (Component II): {ppl_c2:.2f}")

print("\n" + "=" * 65)
print("  COMPARISON: Baseline vs Fine-Tuned — Recipe Instructions")
print("=" * 65)

comparison_c2 = []
for prompt in RECIPE_PROMPTS:
    ft_out = generate_text(model_c2, tokenizer_c2, prompt)
    comparison_c2.append({
        "prompt":    prompt,
        "baseline":  baseline_c2[prompt],
        "finetuned": ft_out
    })
    print(f"\n  Prompt   : {prompt}")
    print(f"  Baseline  : {baseline_c2[prompt][:140]}")
    print(f"  Fine-Tuned: {ft_out[:140]}")

# Save model
model_save_path_c2 = f"{BASE_PATH}/component2_recipes/model"
model_c2.save_pretrained(model_save_path_c2)
tokenizer_c2.save_pretrained(model_save_path_c2)
print(f"\n  Recipe model saved → {model_save_path_c2}")

save_outputs_to_drive(
    {
        "perplexity":  round(ppl_c2, 2),
        "eval_loss":   round(eval_res_c2["eval_loss"], 4),
        "comparisons": comparison_c2
    },
    "c2_finetuned_recipes.json",
    f"{BASE_PATH}/component2_recipes/outputs"
)
```

```
<IPython.core.display.HTML object>

 Perplexity (Component II): 36.75

====================================================================
  COMPARISON: Baseline vs Fine-Tuned — Recipe Instructions
====================================================================

 Prompt   : To make butter chicken
   Baseline  : To make butter chicken with sugar and water, add the
onions in a small saucepan. Bring to boil then saute for 2-3 minutes
until tender (abou
   Fine-Tuned: To make butter chicken add turmeric chili powder and
garam masala. Add ginger garlic paste, sesame oil & sugar then cook
for one minute unti

 Prompt   : For pasta carbonara
   Baseline  : For pasta carbonara, we prefer the whole-wheat method.
This allows us to use less salt than a regular cup of hot water and
also for better r
   Fine-Tuned: For pasta carbonara boil spaghetti in salted water
until al dente and reserve half cup of warm milk. Remove from heat
then gently stir toget

 Prompt   : To prepare a chocolate cake
   Baseline  : To prepare a chocolate cake for the oven, you will
need to find your pie crust. I used my own dough from our homemade
bakery and it was very
   Fine-Tuned: To prepare a chocolate cake mix add sugar and vanilla
extract. Line baking sheet with parchment paper then gently stir in
egg mixture onto t

 Prompt   : The first step in making stir fry is
   Baseline  : The first step in making stir fry is to make it
yourself and whisk until thick. Once the mixture has reached a simmer,
add garlic powder (or
   Fine-Tuned: The first step in making stir fry is to add the soy
sauce and ginger garlic paste. Add half cup water then quickly remove
from heat, cover w
```

{"model_id":"192ca695d85f41fdb56824b9467ca560","version_major":2,"version_minor":0}

```
 Recipe model saved → 
/content/drive/MyDrive/CSET419_Lab11/component2_recipes/model
   Saved → 
/content/drive/MyDrive/CSET419_Lab11/component2_recipes/outputs/c2_finetuned_recipes.json
```

{"type":"string"}

```
# ═══════════════════════════════════════════════════════════════
# CELL 13 — Final Summary Report & Drive Export
# ═══════════════════════════════════════════════════════════════

report = {
    "lab": "CSET419 Lab 11 — Fine-Tuning GPT-2",
    "timestamp": datetime.now().isoformat(),
    "device": DEVICE.upper(),
    "framework": "PyTorch + Hugging Face Transformers",
    "component_1": {
        "task":            "Product Review Generator (E-Commerce)",
        "model":           "GPT-2 (124M params)",
        "dataset_size":    20,
        "train_epochs":    15,
        "learning_rate":  5e-5,
        "perplexity":      round(ppl_c1, 2),
        "model_saved_at": model_save_path_c1,
        "observation":    "Fine-tuned model generates e-commerce
review style text",
    },
    "component_2": {
        "task":            "Recipe Instruction Generator (Food-Tech)",
        "model":           "GPT-2 (124M params)",
        "dataset_size":    20,
        "train_epochs":    15,
        "learning_rate":  5e-5,
        "perplexity":      round(ppl_c2, 2),
        "model_saved_at": model_save_path_c2,
        "observation":    "Fine-tuned model generates step-by-step
cooking instructions",
    },
}

# Save report
report_path = save_outputs_to_drive(
    report, "lab11_final_report.json",
    f"{BASE_PATH}/reports"
)

# ── Pretty Print Summary ──────────────────────────────────
print("\n" + "═" * 60)
print("  LAB 11 COMPLETE — FINAL SUMMARY")
print("═" * 60)
print(f"""
  ┌──────────────────────────────────────────────────────┐
  │  COMPONENT I  :  Product Review Generator            │
  │  Perplexity   :  {ppl_c1:.2f}  (lower = better)          │
  │  Result       :  □ Review-style text generated       │
  ├──────────────────────────────────────────────────────┤
  │  COMPONENT II :  Recipe Instruction Generator        │
```

```
    │  Perplexity   :  {ppl_c2:.2f}  (lower = better)          │
    │  Result       :  □ Step-by-step cooking generated        │
    ├──────────────────────────────────────────────────────────┤
    │  Drive Folder : MyDrive/CSET419_Lab11/                   │
    │  ├── component1_reviews/model/   (saved model)           │
    │  ├── component1_reviews/outputs/ (JSON results)          │
    │  ├── component2_recipes/model/   (saved model)           │
    │  ├── component2_recipes/outputs/ (JSON results)          │
    │  └── reports/lab11_final_report.json                     │
    └──────────────────────────────────────────────────────────┘
""")
print("□ All outputs successfully saved to Google Drive!")
```

```
  □ Saved →
/content/drive/MyDrive/CSET419_Lab11/reports/lab11_final_report.json

═══════════════════════════════════════════════════════════════════
  LAB 11 COMPLETE — FINAL SUMMARY
═══════════════════════════════════════════════════════════════════

    ┌──────────────────────────────────────────────────────────┐
    │  COMPONENT I  :  Product Review Generator                │
    │  Perplexity   :  24.74  (lower = better)                 │
    │  Result       :  □ Review-style text generated           │
    ├──────────────────────────────────────────────────────────┤
    │  COMPONENT II :  Recipe Instruction Generator            │
    │  Perplexity   :  36.75  (lower = better)                 │
    │  Result       :  □ Step-by-step cooking generated        │
    ├──────────────────────────────────────────────────────────┤
    │  Drive Folder : MyDrive/CSET419_Lab11/                   │
    │  ├── component1_reviews/model/   (saved model)           │
    │  ├── component1_reviews/outputs/ (JSON results)          │
    │  ├── component2_recipes/model/   (saved model)           │
    │  ├── component2_recipes/outputs/ (JSON results)          │
    │  └── reports/lab11_final_report.json                     │
    └──────────────────────────────────────────────────────────┘

□ All outputs successfully saved to Google Drive!
```